Question 1: Story and MVP

1. The model, view, and presenter

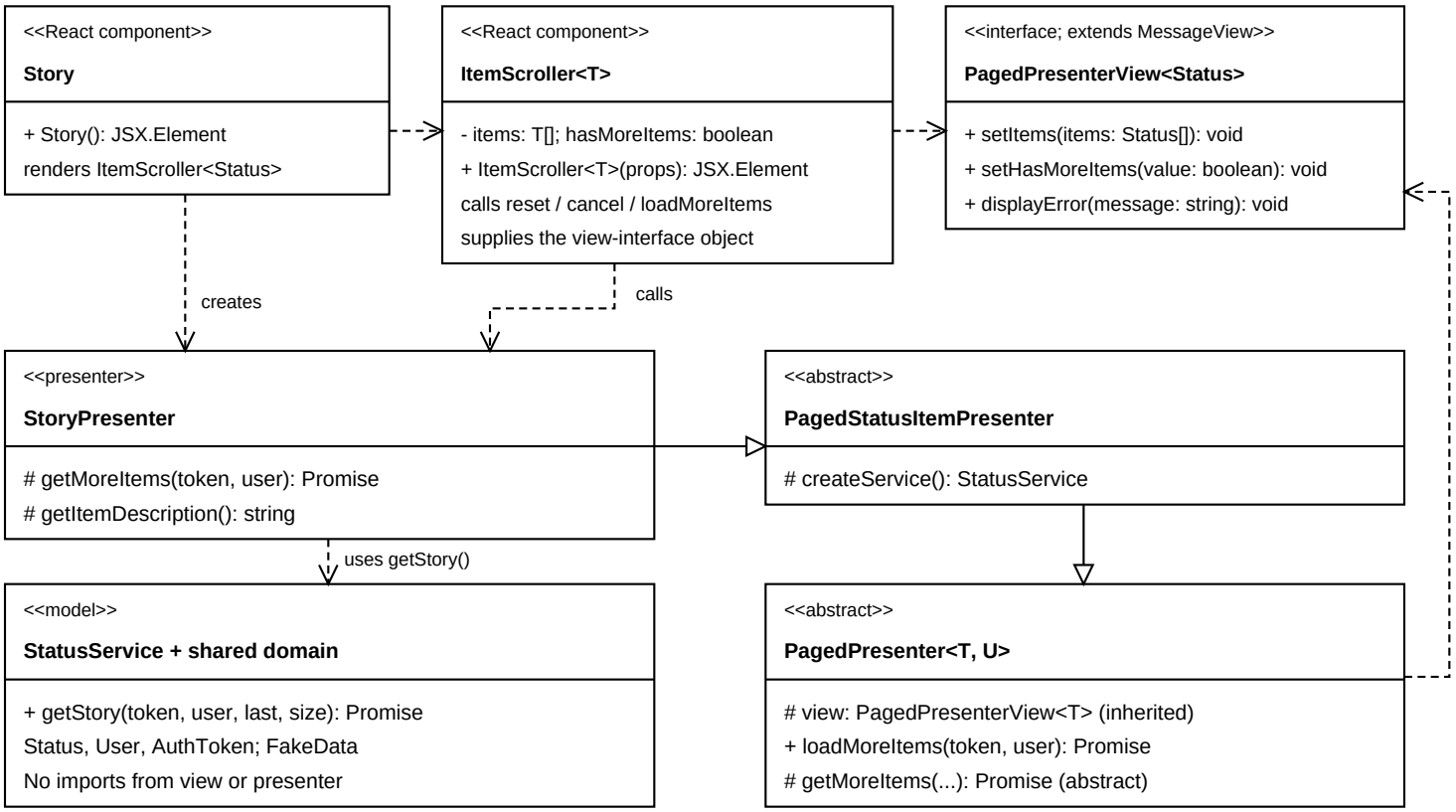
In my design, Story and ItemScroller<Status> make up the view. Story creates the StoryPresenter and uses StatusItem to display each status. ItemScroller keeps the items and hasMoreItems in React state. It gives the presenter a PagedPresenterView<Status> object with methods for updating the list, setting whether more items are available, and showing an error.

StoryPresenter is the presenter for the story. It extends PagedStatusItemPresenter, which extends PagedPresenter<Status, StatusService>. The parent class handles loading pages of items. StoryPresenter's getMoreItems method calls StatusService.getStory with the auth token, user, last item, and page size. Its getItemDescription method returns "story" for error messages. Each page has up to 10 statuses.

The model includes StatusService and the shared Status, User, AuthToken, and FakeData types. StatusService gets the story data. For this milestone, getStory calls getFeed, which returns hardcoded statuses from FakeData. The app does not save new statuses or make a real backend request yet.

The view uses the presenter, and the presenter uses the model. To update the screen, the presenter calls methods on the PagedPresenterView interface. It does not import the React components or know how they display the data. The view provides the methods that implement this interface. The model also does not import the view or presenter. This follows the dependency rule because the lower layers do not depend on the UI code. Calling a view method through an interface does not require the presenter to depend on a specific React component.

Figure 1. Story class and component relationships



Hollow triangles: inheritance / interface realization. Dashed open arrows: dependencies.

Question 1: Story interaction

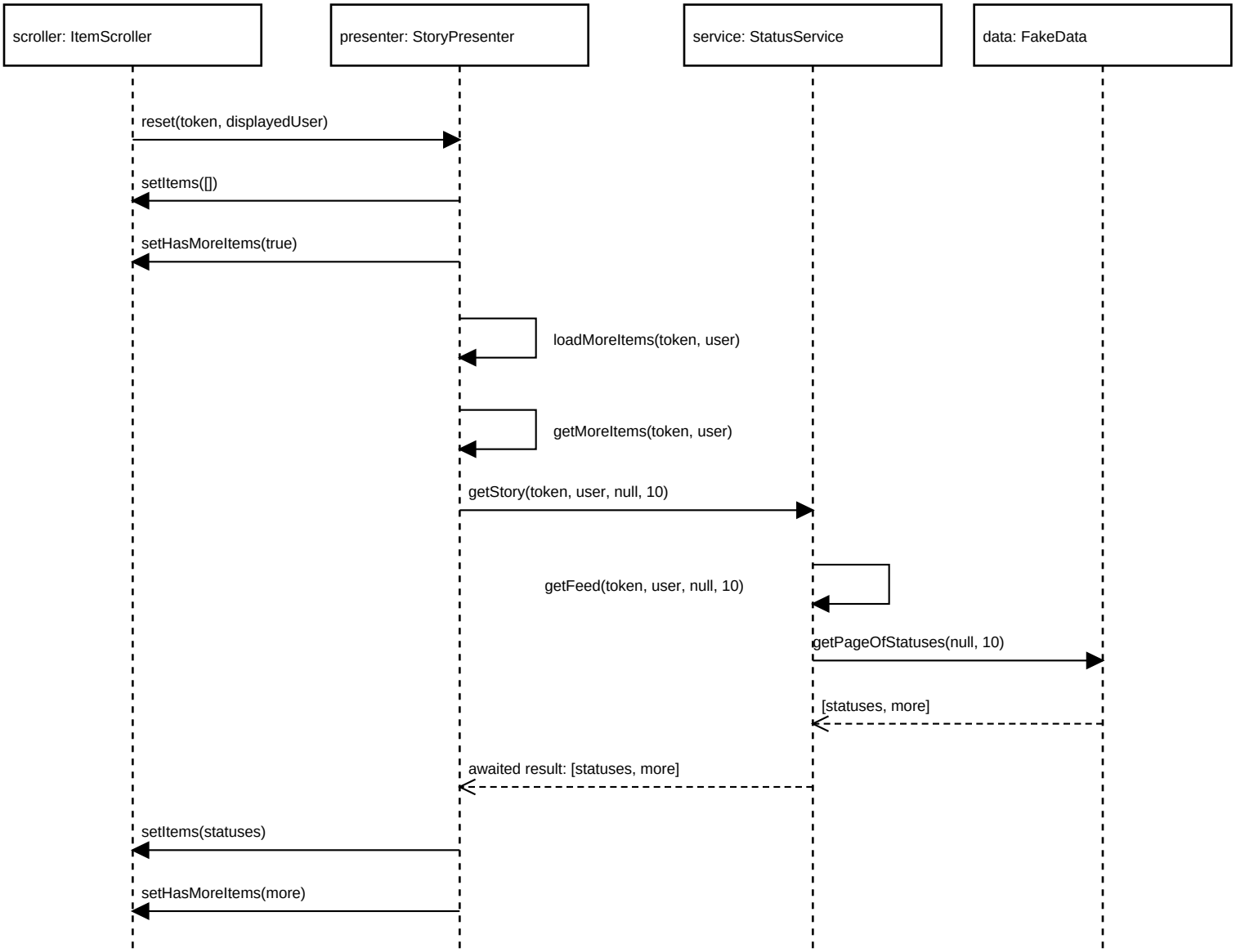
2. Why MVP works well for my design

1. It separates the jobs of the different parts. The view displays the statuses, the presenter handles loading them, and the service provides the data. This should make it easier to replace the dummy data with a backend later.

2. It makes testing easier. I can mock a presenter's view and service to test its behavior without rendering a React component. This is how the navbar and posting presenter tests work.

3. It lets me reuse code. StoryPresenter and FeedPresenter share the page-loading steps but call different service methods. Both views can also use ItemScroller to display their lists.

Figure 2. First story page: sequence diagram



Solid arrows are ordinary calls, including methods that return Promises. The dashed service-to-presenter arrow summarizes the awaited result, not an immediate return. At await, the continuation pauses without blocking the browser. reset does not await loadMoreItems. Only a successful first page is shown; internal helpers and error/cancel paths are omitted.

Question 2: Template Method

A. How the classes work together / B. How they share code

A. The superclass and subclasses

I used Template Method in the paged presenters. PagedPresenter<T, U> is the abstract superclass, and loadMoreItems is the template method. It defines the common steps: check whether another page can be loaded, call getMoreItems, add the results to the list, remember the last item, and update the view. It also makes sure an old request does not update a different user's list. Error handling comes from doFailureReportingOperation in ErrorReportingPresenter.

The subclasses provide the parts that change. StoryPresenter implements getMoreItems by calling getStory, while FeedPresenter calls getFeed. They also implement getItemDescription so errors refer to the right list. These methods are the hooks used by the template method. PagedStatusItemPresenter is their shared parent and implements createService to provide a StatusService. Both concrete presenters use the lastItem and pageSize from PagedPresenter.

B. Reducing duplicate code

Before Part B, ScrollerPresenter<T> already shared the paging state, the loadMore method, and paging error handling between story and feed. StoryPresenter and FeedPresenter each had a similar constructor. Each constructor created a StatusService and passed the parent a label and a service method to call. Figure 3 shows this earlier design.

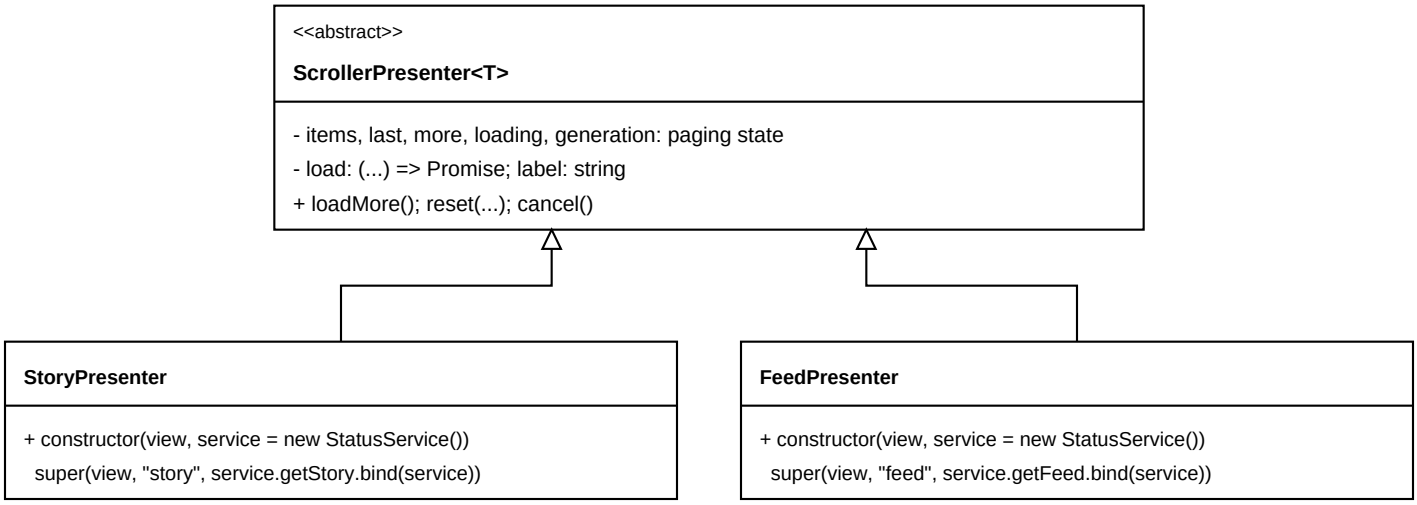
After the refactor, PagedPresenter.loadMoreItems contains the shared steps and calls the methods that the subclasses override. PagedStatusItemPresenter creates the StatusService in one place instead of repeating that setup in both constructors. ErrorReportingPresenter extends error-reporting reuse across different presenter types, rather than just the paged lists. Figure 4 shows how StoryPresenter and FeedPresenter inherit the common behavior and supply their own service calls.

The paging loop and its error handling were already shared before this change. The refactor keeps that reuse, shares service creation, and makes the common error-reporting method available to other presenter types. Template Method makes it clearer which steps stay the same and which parts each subclass must provide. For example, I can change how pages are added to the list in PagedPresenter without changing both StoryPresenter and FeedPresenter.

Question 2: Before and after

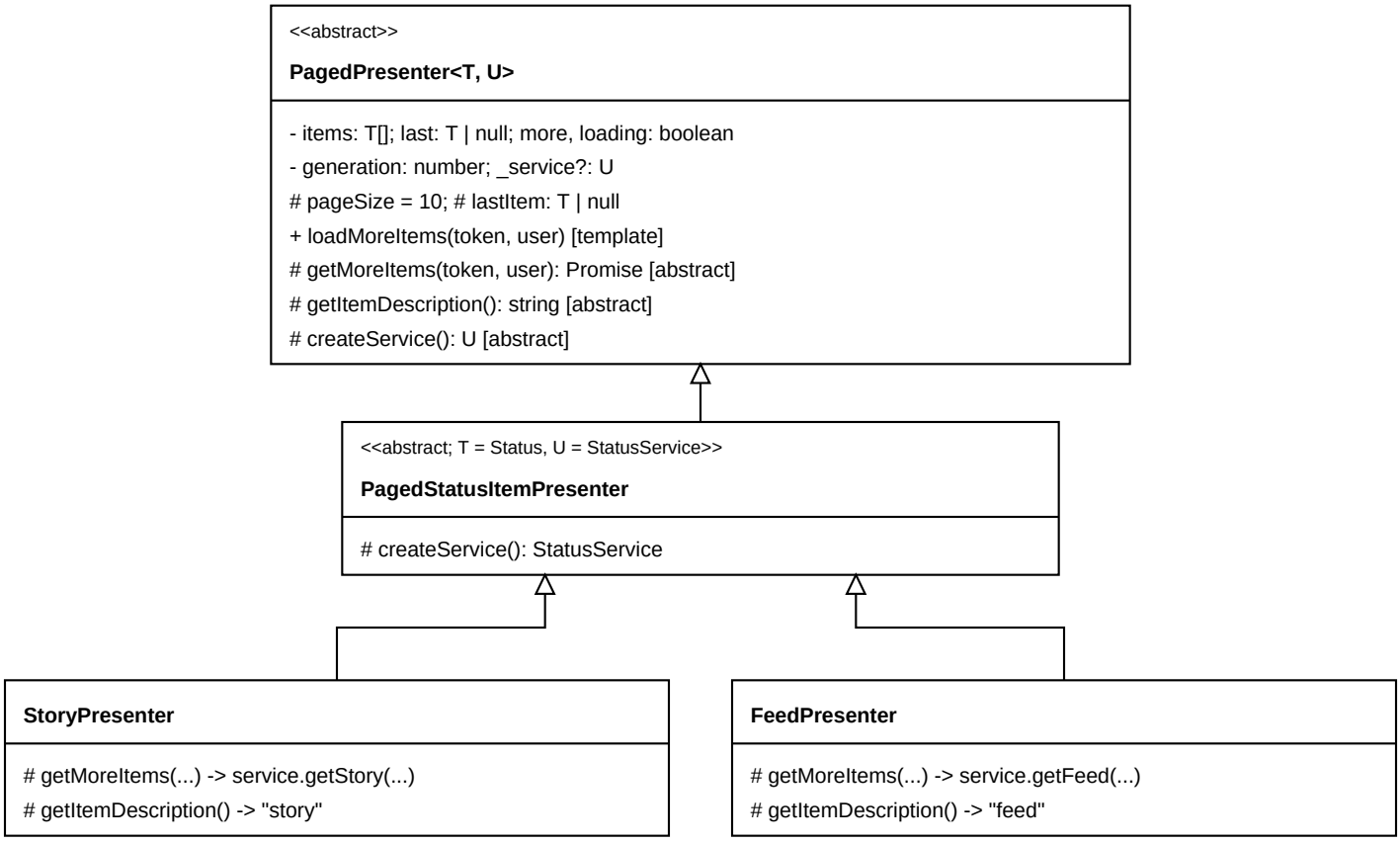
The design before Part B and the current design

Figure 3. Before



The parent already handled paging. Each subclass created a StatusService and passed the parent a label and the service method to use.

Figure 4. After



Hollow triangles point to the superclass. Protected (#) hooks vary the fixed template (+).